TemporalDB Backend — Architecture & Local Setup Guide

Table of Contents

1. System Overview
2. Technology Stack
3. High-Level Architecture
4. Project Structure
5. Component Deep-Dives
6. 5.1 Application Entry Point
7. 5.2 Configuration
8. 5.3 Database Layer
9. 5.4 Data Models
10. 5.5 Ingestion Pipeline
11. 5.6 NLP Pipeline
12. 5.7 Storage Layer
13. 5.8 Query Engine
14. 5.9 LLM Integration
15. 5.10 API Layer
16. 5.11 NLP Worker
17. Database Schema
18. API Reference
19. Data Flow Walkthroughs
20. Running Locally

1. System Overview

TemporalDB is a backend system that ingests unstructured text documents (reports, articles, transcripts) and extracts a structured, queryable representation of the **events**, **entities**, and **causal relationships** contained within them.

Once ingested, the data can be queried in natural language. The engine classifies the intent of a question, executes the appropriate retrieval strategy, and synthesises a prose answer backed by citations.

Core capabilities:

Capability	Description
Document ingestion	PDF, DOCX, TXT, Markdown via REST upload or raw text
Deduplication	SHA-256 fingerprint checked against Redis before processing
NLP extraction	Entities, events, temporal expressions, causal relations
Graph persistence	All events and causal links mirrored to Neo4j
Natural language query	Four query strategies (causal, temporal, similarity, entity)
LLM synthesis	Ollama-backed prose answers with fallback templates

2. Technology Stack

Layer	Technology	Version
API framework	FastAPI + Uvicorn	0.115.6 / 0.34.0
Relational DB	PostgreSQL + pgvector	16
Graph DB	Neo4j Community	5
Cache / dedup	Redis	7
Message bus	Redpanda (Kafka-compatible)	latest
ORM	SQLAlchemy (async) + asyncpg	2.0.36 / 0.30.0
Migrations	Alembic	1.14.1
Validation	Pydantic v2	2.10.4
NLP	spaCy	3.8.3
Embeddings	sentence-transformers (all-MiniLM-L6-v2)	3.3.1
Fuzzy matching	rapidfuzz	3.11.0
Temporal parsing	dateparser	1.2.1
Document parsing	PyMuPDF (PDF), python-docx (DOCX)	1.25.3 / 1.1.2
LLM	Ollama (llama3.1:8b / codellama:7b)	local
Task queue	Celery + Redis	5.4.0
Logging	structlog (JSON)	24.4.0
Container runtime	Docker / Docker Compose	—

3. High-Level Architecture

```
Client (curl / frontend / SDK)
         |
         | HTTP  X-API-Key auth
         ▼
┌──────────────────────────────────────────────────────────┐
│                FastAPI Application  :8000                 │
│                                                           │
│  POST /ingest          POST /query        GET /events     │
│  POST /ingest/file     GET /health        GET /entities   │
│                                           GET /graph/...  │
└──────────┬──────────────────┬──────────────────────┬──────┘
           │                  │                      │
           ▼                  ▼                      ▼
 ┌─────────────────┐  ┌──────────────┐  ┌──────────────────┐
 │ Ingestion Layer │  │ Query Engine │  │  Browse / Graph  │
 │ - normalize     │  │ - intent class│  │  event_store     │
 │ - deduplicate   │  │ - time extract│  │  entity_store    │
 │ - persist to PG │  │ - entity res. │  │  graph_store     │
 │ - publish Kafka │  │ - planner     │  └──────────────────┘
 └────────┬────────┘  │ - synthesize  │
          │           └──────┬────────┘
          │                  │
          ▼                  │ reads
 ┌────────────────┐          │
 │  Kafka Topic   │   ┌──────▼──────────────────────────┐
 │ document.      │   │            Planners              │
 │   ingested     │   │  CausalPlanner  → Neo4j BFS      │
 └────────┬───────┘   │  TemporalPlanner → PG date filter │
          │           │  SimilarityPlanner → pgvector ANN │
          ▼           │  EntityPlanner  → PG + Neo4j     │
 ┌────────────────┐   └──────────────────────────────────┘
 │   NLP Worker   │
 │  (Kafka consum.)│          ┌──────────┐   ┌──────────┐
 │  coref → NER   │──writes──►│PostgreSQL│   │  Neo4j   │
 │  temporal      │          │ pgvector │   │  graph   │
 │  events        │          └──────────┘   └──────────┘
 │  entities      │
 │  causal links  │          ┌──────────┐   ┌──────────┐
 └────────────────┘          │  Redis   │   │  Ollama  │
                             │  dedup + │   │  LLM     │
                             │  cache   │   │  :11434  │
                             └──────────┘   └──────────┘
```

Request lifecycle — Ingestion

```
POST /ingest
    │
    ├─ normalize text (Unicode NFC, whitespace, control chars)
    ├─ compute SHA-256 fingerprint
    ├─ check Redis  → if duplicate, return 200 {status: "duplicate"}
    ├─ persist Document row to PostgreSQL
    ├─ publish "document.ingested" to Kafka topic
    └─ return 202 {status: "processing"}

Kafka consumer (nlp_worker):
    │
    ├─ coref resolution      (pronoun → antecedent)
    ├─ NER                   (spaCy entities)
    ├─ temporal parsing      (dates → UTC TemporalSpans)
    ├─ event extraction      (SVO dependency parsing)
    ├─ entity linking        (exact / fuzzy / embedding tiers)
    ├─ causal extraction     (cue phrases + dependency patterns)
    ├─ persist Events, Entities, CausalRelations to PostgreSQL
    └─ sync to Neo4j         (Event/Entity nodes, CAUSES/INVOLVES edges)
```

Request lifecycle — Query

```
POST /query  { "question": "Why did revenue fall in Q3 2024?" }
    │
    ├─ classify intent       → CAUSAL_WHY
    ├─ extract time range    → TimeRange(2024-07-01, 2024-09-30)
    ├─ resolve entity filter → None (no entity mention in question)
    ├─ run CausalPlanner:
    │      embed question → similarity_search seeds
    │      graph_store.get_causal_chain(seed, max_hops=3)
    │      fetch extra events from PostgreSQL
    │      confidence = min(0.90, 0.70 + 0.10 × chain_length)
    ├─ synthesizer:
    │      fetch source Documents
    │      call Ollama llama3.1:8b  (or fallback template)
    └─ return QueryResponse { answer, confidence, intent,
                              causal_chain, events, sources }
```

4. Project Structure

```
TemporalDB Backend/
│
├── app/                          Main application package
│   ├── main.py                   FastAPI app, lifespan, middleware, routers
│   ├── config.py                 Pydantic Settings — all env vars with defaults
│   │
│   ├── api/                      HTTP layer
│   │   ├── middleware.py         RequestLoggingMiddleware (structlog)
│   │   └── routes/
│   │       ├── __init__.py       require_api_key() shared dependency
│   │       ├── ingest.py         POST /ingest, POST /ingest/file
│   │       ├── events.py         GET /events, GET /events/{id}
│   │       ├── entities.py       GET /entities, GET /entities/{id}
│   │       ├── graph.py          GET /graph/entity/{id}
│   │       └── query.py          POST /query
│   │
│   ├── database/                 Connection lifecycle managers
│   │   ├── postgres.py           SQLAlchemy async engine + get_db dependency
│   │   ├── neo4j.py              Neo4j driver + get_neo4j dependency
│   │   └── redis.py              Redis pool + get_redis dependency
│   │
│   ├── models/
│   │   ├── sql/                  SQLAlchemy ORM models
│   │   │   ├── document.py       Document (id, source, content_hash, raw_text …)
│   │   │   ├── event.py          Event (id, description, embedding, ts_start …)
│   │   │   ├── entity.py         Entity (id, canonical_name, type, aliases …)
│   │   │   ├── event_entity.py   Junction table: event_id ↔ entity_id
│   │   │   └── causal_relation.py CausalRelation (cause_event_id, effect_event_id …)
│   │   └── schemas/              Pydantic request / response schemas
│   │       ├── ingest.py         TextIngestRequest, IngestResponse
│   │       ├── event.py          EventOut, EventBrief, EventListResponse
│   │       ├── entity.py         EntityOut, EntityBrief, EntityListResponse
│   │       ├── query.py          QueryRequest, TimeRange, QueryResponse …
│   │       └── graph.py          GraphNode, GraphEdge, GraphResponse
│   │
│   ├── ingestion/                Pre-NLP ingestion stages
│   │   ├── connectors/
│   │   │   ├── base.py           BaseConnector ABC, ConnectorResult dataclass
│   │   │   └── file.py           FileConnector (PDF via PyMuPDF, DOCX, TXT, MD)
│   │   ├── normalizer.py         normalize(text) → clean Unicode string
│   │   ├── deduplicator.py       compute_fingerprint(), check_and_register()
│   │   └── producer.py           Kafka producer, publish_document_ingested()
│   │
│   ├── nlp/                      NLP pipeline stages
│   │   ├── pipeline.py           run_pipeline_sync() / run_pipeline() orchestrator
│   │   ├── coref.py              resolve_coref_sync() — pronoun coreference
│   │   ├── ner.py                extract_entities_sync() — spaCy NER
│   │   ├── temporal_parser.py    parse_temporal_expressions() → TemporalSpan[]
│   │   ├── event_extractor.py    extract_events_sync() — SVO dependency parsing
│   │   ├── entity_linker.py      link_entities() — 3-tier clustering
│   │   ├── causal_extractor.py   extract_causal_relations() — cue + dependency
│   │   └── embedder.py           embed() / embed_batch() — sentence-transformers
│   │
│   ├── storage/                  Database persistence (called from NLP worker)
│   │   ├── event_store.py        insert_event, list_events, similarity_search …
│   │   ├── entity_store.py       upsert_entity, bulk_upsert_entities …
```

```
│   │   ├── graph_store.py        Neo4j CRUD + causal chain traversal
│   │   └── sync.py               sync_document() PG → Neo4j bridge
│   │
│   ├── query/                    Query execution engine
│   │   ├── orchestrator.py       handle_query() — main dispatcher
│   │   ├── intent.py             classify_intent() — heuristic + LLM fallback
│   │   ├── temporal_extractor.py extract_time_range() from question text
│   │   ├── entity_resolver.py    resolve_entity_filter() mention → UUID
│   │   ├── synthesizer.py        synthesize() → QueryResponse (Ollama + fallback)
│   │   └── planners/
│   │       ├── __init__.py       PlanResult dataclass
│   │       ├── causal_planner.py Neo4j BFS from seed events
│   │       ├── temporal_planner.py PG date-bounded list_events
│   │       ├── similarity_planner.py pgvector ANN search
│   │       └── entity_planner.py PG events + Neo4j entity subgraph
│   │
│   └── llm/                      Local LLM integration
│       ├── client.py             OllamaClient (generate, chat, embed, …)
│       └── prompts.py            Prompt templates (intent, synthesis, extraction …)
│
├── workers/
│   └── nlp_worker.py             Kafka consumer — runs the NLP pipeline per message
│
├── alembic/                      Database migrations
│   ├── env.py                    Reads postgres_dsn from Settings
│   ├── script.py.mako            Migration file template
│   └── versions/
│       └── c73d4a0b0c3e_initial_tables.py   Initial schema migration
│
├── tests/
│   ├── unit/                     Fully mocked unit tests (no infrastructure)
│   ├── integration/              Tests requiring running Docker services
│   └── api/                      FastAPI TestClient endpoint tests
│
├── Dockerfile                    Multi-stage production image
├── .dockerignore
├── docker-compose.yml            Local infrastructure (PG, Neo4j, Redis, Redpanda)
├── Makefile                      Developer command suite
├── requirements.txt              All deps including dev/test tools
├── requirements-prod.txt         Production deps only (used in Dockerfile)
├── alembic.ini                   Alembic config
├── pytest.ini                    pytest asyncio_mode = auto
└── .env.example                  Environment variable template
```

---

5. Component Deep-Dives

5.1 Application Entry Point — `app/main.py`

FastAPI application with an `asynccontextmanager` lifespan that initialises and tears down all service connections in order:

Startup sequence:

1. `init_postgres()` — create SQLAlchemy async engine and run a test query

2. `init_neo4j()` — open Neo4j driver, verify connectivity
3. `init_redis()` — create Redis connection pool
4. `init_kafka_producer()` — create aiokafka producer, await metadata

Shutdown sequence (reverse order):
`close_kafka_producer()` → `close_redis()` → `close_neo4j()` → `close_postgres()`

Middleware stack (outermost to innermost):
1. `RequestLoggingMiddleware` — logs method, path, status, duration via structlog
2. `CORSMiddleware` — allow_origins=["*"] (tighten for production)

Exception handler: catches all unhandled exceptions, returns structured `{"error", "detail", "code": "INTERNAL_ERROR"}` JSON. In debug mode the full exception message is included; in production a generic message is used.

5.2 Configuration — `app/config.py`

Single `Settings` class (Pydantic `BaseSettings`) loaded from `.env` file at startup. All values have sensible defaults that match the `docker-compose.yml` port mappings.

Setting group	Key env vars
App	`DEBUG`, `API_KEY`
PostgreSQL	`POSTGRES_HOST/PORT/USER/PASSWORD/DB`
Neo4j	`NEO4J_URI`, `NEO4J_USER`, `NEO4J_PASSWORD`
Redis	`REDIS_HOST`, `REDIS_PORT`, `REDIS_DB`, `REDIS_PASSWORD`
Kafka	`KAFKA_BOOTSTRAP_SERVERS`, `KAFKA_INGESTION_TOPIC`, `KAFKA_CONSUMER_GROUP`
Ollama	`OLLAMA_BASE_URL`, `OLLAMA_SYNTHESIS_MODEL`, `OLLAMA_CODEGEN_MODEL`
NLP	`SPACY_MODEL`, `EMBEDDING_MODEL`, `EMBEDDING_DIMENSION`
Server	`HOST`, `PORT`

Computed properties: `postgres_dsn` (async asyncpg URL), `postgres_dsn_sync` (sync URL for Alembic), `redis_url`, `celery_broker_url`, `celery_result_backend`.

5.3 Database Layer — `app/database/`

Each module follows the same pattern: a module-level client/engine variable, `init_*` / `close_*` coroutines called by `main.py` lifespan, and a FastAPI dependency generator used in route handlers.

Module	Client	Dependency
`postgres.py`	`async_session_factory` (SQLAlchemy)	`get_db()` → `AsyncSession`
`neo4j.py`	`_driver` (Neo4j `AsyncDriver`)	`get_neo4j()` → `AsyncSession`
`redis.py`	`_redis` (redis-py `Redis`)	`get_redis()` → `Redis`

`postgres.py` also defines `Base` (SQLAlchemy `DeclarativeBase`) imported by all ORM models.

5.4 Data Models — `app/models/`

PostgreSQL ORM (`app/models/sql/`)

```
documents
├── id              UUID  PK
├── source          TEXT  (URL, file path, or label)
├── filename        TEXT  nullable
├── content_hash    TEXT  (SHA-256, unique index)
├── raw_text        TEXT
├── mime_type       TEXT  nullable
├── metadata_       JSONB nullable
└── ingested_at     TIMESTAMPTZ  default now()

events
├── id              UUID  PK
├── description     TEXT
├── event_type      TEXT  nullable  (action, state_change, …)
├── ts_start        TIMESTAMPTZ nullable
├── ts_end          TIMESTAMPTZ nullable
├── confidence      FLOAT  default 1.0
├── source_sentence TEXT  nullable
├── embedding       VECTOR(384)  nullable  [pgvector]
└── document_id     UUID  FK → documents.id

entities
├── id              UUID  PK
├── name            TEXT  (raw mention text)
├── canonical_name  TEXT  (resolved form, unique index)
├── type            TEXT  (PERSON, ORG, GPE, …)
├── description     TEXT  nullable
├── aliases         JSONB nullable  (list of alternate names)
└── created_at      TIMESTAMPTZ  default now()

event_entities                    (junction)
├── event_id        UUID  FK → events.id
└── entity_id       UUID  FK → entities.id

causal_relations
├── id              UUID  PK
├── cause_event_id  UUID  FK → events.id
├── effect_event_id UUID  FK → events.id
├── confidence      FLOAT  default 1.0
├── evidence        TEXT  nullable  (triggering phrase)
└── created_at      TIMESTAMPTZ  default now()
```

Indexes: pgvector HNSW index on `events.embedding`; B-tree on `events.document_id`, `entities.canonical_name`, `causal_relations.cause_event_id` and `effect_event_id`.

Neo4j Schema

```
Nodes
  (:Event)   { id, description, event_type, ts_start, ts_end,
               confidence, source_sentence, document_id }
  (:Entity)  { id, name, canonical_name, type }

Relationships
  (:Event)-[:CAUSES { relation_id, confidence, evidence }]->(:Event)
  (:Event)-[:INVOLVES]->(:Entity)
```

All writes use `MERGE` on `id` / `relation_id` so they are fully idempotent.

Pydantic Schemas (`app/models/schemas/`)

Schema module	Key types
`ingest.py`	`TextIngestRequest` , `IngestResponse`
`event.py`	`EventOut` , `EventBrief` , `EventListResponse`
`entity.py`	`EntityOut` , `EntityBrief` , `EntityListResponse`
`query.py`	`QueryRequest` , `TimeRange` , `CausalChainLink` , `SourceReference` , `QueryResponse`
`graph.py`	`GraphNode` , `GraphEdge` , `GraphResponse`

All `Out` schemas use `model_config = ConfigDict(from_attributes=True)` for direct ORM-to-schema validation.

5.5 Ingestion Pipeline — `app/ingestion/`

`connectors/base.py`

Defines `BaseConnector` ABC with `extract(source) → ConnectorResult` and `ConnectorResult(text, mime_type, filename, metadata)` .

`connectors/file.py`

`FileConnector` dispatches on file extension:

Extension	Library	Notes
`.pdf`	PyMuPDF (`fitz`)	Extracts all page text
`.docx`	python-docx	Joins all paragraph text
`.txt` , `.md`	built-in	UTF-8 read

`normalizer.py`

Pure function `normalize(text: str) → str` :

1. Unicode NFC normalisation
2. Remove null bytes and non-printable control characters
3. Normalise line endings to `\n`
4. Collapse tab runs to single space
5. Collapse intra-line whitespace runs
6. Collapse 3+ consecutive blank lines to one
7. Strip leading/trailing whitespace

`deduplicator.py`

- `compute_fingerprint(text)` → SHA-256 hex string
- `check_and_register(redis, text)` → `DeduplicationResult(is_duplicate, existing_id)`
- Redis key: `dedup:doc:<fingerprint>`, value: document UUID
- Sets key with no expiry on first registration

`producer.py`

`aiokafka` async producer. Key functions:
- `init_kafka_producer()` / `close_kafka_producer()` — lifecycle
- `publish(topic, key, value)` — low-level send
- `publish_document_ingested(producer, document_id, source)` — publishes to `document.ingested` topic; message body contains `document_id` and `source`

5.6 NLP Pipeline — `app/nlp/`

The pipeline is orchestrated by `pipeline.py` and runs as a linear chain of six stages. The NLP worker calls `run_pipeline_sync()` or the async wrapper `run_pipeline()`.

```
Input text
    |
    ▼
1. coref.resolve_coref_sync(text)
        Pronoun → antecedent substitution using spaCy dependency
        morphology (3rd-person singular + plural, relative pronouns).
        Output: resolved_text (str)
    |
    ▼
2. ner.extract_entities_sync(resolved_text)
        spaCy NER over RELEVANT_ENTITY_TYPES
        (PERSON, ORG, GPE, DATE, MONEY, PERCENT, EVENT, …).
        Output: List[NEREntity(text, label, start, end)]
    |
    ▼
3. temporal_parser.parse_temporal_expressions(resolved_text)
        Pattern + dateparser extraction of date/time spans.
        Handles quarters (Q1–Q4), relative expressions, ISO dates.
        All times normalised to UTC.
        Output: List[TemporalSpan(text, start_dt, end_dt, raw)]
    |
    ▼
4. event_extractor.extract_events_sync(resolved_text, temporal_spans)
        spaCy dependency parse → Subject-Verb-Object triples.
        Attaches the closest sentence-scoped TemporalSpan.
        Collects modifiers (prep, advmod, prt children of verb).
        Output: List[ExtractedEvent(subject, verb, object, modifiers,
                                    temporal_span, source_sentence)]
    |
    ▼
5. entity_linker.link_entities(ner_entities, resolved_text)
        Three-tier intra-document clustering:
          Tier 1 — exact text match        (confidence 1.0)
          Tier 2 — fuzzy overlap ≥ 0.85   (rapidfuzz)
          Tier 3 — embedding similarity ≥ 0.92 (sentence-transformers)
        Assigns cluster_id and canonical_name to each mention.
        Output: List[LinkedEntity(text, label, cluster_id, canonical_name)]
    |
    ▼
6. causal_extractor.extract_causal_relations(resolved_text, events)
        Two extraction strategies:
          a) Lexical cue phrases: "because", "due to", "led to",
             "caused by", "as a result of", "resulted in", …
          b) Dependency patterns: causal adverbial clauses (advcl),
             prepositional objects of causal prepositions.
        Output: List[CausalRelation(cause, effect, confidence, evidence)]
    |
    ▼
PipelineResult(
    resolved_text, entities, temporal_spans,
    events, linked_entities, causal_relations
)
```

`embedder.py`

Wraps `sentence_transformers.SentenceTransformer('all-MiniLM-L6-v2')`.
Produces 384-dimensional L2-normalised float vectors.

Functions: `embed(text)`, `embed_batch(texts)` and their `_sync` equivalents.

5.7 Storage Layer — `app/storage/`

Called by the NLP worker after pipeline completion to persist results.

`event_store.py`

Function	Description
`insert_event(session, **kwargs)`	INSERT event row, returns ORM object
`get_event_by_id(session, id)`	SELECT by PK, returns Event or None
`list_events(session, ...)`	Paginated SELECT with optional filters: `entity_id`, `document_id`, `from_date`, `to_date`, `event_type`. Returns `(events, total)`
`delete_event(session, id)`	DELETE by PK, returns bool
`link_entities_to_event(session, event_id, entity_ids)`	Bulk INSERT into `event_entities`, skips duplicates
`insert_causal_relation(session, ...)`	INSERT causal_relation row
`get_causal_relations(session, event_id)`	SELECT all relations where event is cause or effect
`similarity_search(session, embedding, k, entity_id?)`	pgvector ANN query: `ORDER BY embedding <-> $vec LIMIT k` with optional entity filter

`entity_store.py`

Function	Description
`upsert_entity(session, neo4j_session?, **kwargs)`	INSERT or UPDATE by `canonical_name`. Optionally mirrors to Neo4j
`bulk_upsert_entities(session, entities)`	Upsert list, returns canonical_name→Entity map with deduplication
`get_entity_by_id(session, id)`	SELECT by PK
`get_entity_by_canonical_name(session, name)`	SELECT by canonical_name
`list_entities(session, ...)`	Paginated SELECT with optional name search and type filter
`delete_entity(session, neo4j_session?, id)`	DELETE from PG and optionally Neo4j
`merge_aliases(entity, new_name)`	Append to `aliases` JSONB list, deduplicates

`graph_store.py`

Function	Description
`upsert_event_node(session, **kwargs)`	`MERGE (:Event {id})` with SET all properties
`upsert_entity_node(session, **kwargs)`	`MERGE (:Entity {id})` with SET all properties
`upsert_causal_edge(session, **kwargs)`	`MERGE (cause)-[:CAUSES {relation_id}]->(effect)`
`upsert_involves_edge(session, event_id, entity_id)`	`MERGE (event)-[:INVOLVES]->(entity)`
`delete_event_node(session, id)`	`MATCH DETACH DELETE`, returns bool
`delete_causal_edge(session, relation_id)`	`MATCH DELETE`, returns bool
`get_causal_chain(session, event_id, direction, max_hops)`	Variable-length path traversal `[:CAUSES*1..N]`. `direction`: `"downstream"`, `"upstream"`, `"both"`. `max_hops` clamped to [1, 10]
`get_entity_graph(session, entity_id, max_events)`	Two queries: (1) all Events with INVOLVES edge to entity; (2) all CAUSES edges among those events
`get_causal_path_between(session, src, tgt, max_hops)`	`shortestPath((src)-[:CAUSES*1..N]->(tgt))`

`sync.py`

`sync_document(pg_session, neo4j_session, document_id) → SyncResult`

Orchestrates a full PG → Neo4j sync for one document:
1. Fetch all Events for the document (with entities eagerly loaded)
2. `upsert_event_node` for each event
3. Collect unique entities across all events; `upsert_entity_node` once per entity
4. Fetch all `EventEntity` links; `upsert_involves_edge` for each
5. Fetch all `CausalRelation` rows; `upsert_causal_edge` for each

Returns `SyncResult(event_nodes, entity_nodes, involves_edges, causal_edges)`.
`SyncResult` supports `+` operator for aggregation and `as_dict()` for logging.

5.8 Query Engine — `app/query/`

`orchestrator.py` — `handle_query(request, pg, neo4j)`

Entry point for all natural language queries:

```
1. classify_intent(question)          → IntentResult(intent, confidence, method)
2. extract_time_range(question)       → TimeRange | None
3. resolve_entity_filter(pg, question)→ UUID | None
4. dispatch to planner based on intent:
     CAUSAL_WHY       → causal_planner.run(pg, neo4j, question, entity_id, max_hops)
     TEMPORAL_RANGE   → temporal_planner.run(pg, time_range, entity_id)
     SIMILARITY       → similarity_planner.run(pg, question, entity_id, time_range, limit)
     ENTITY_TIMELINE  → entity_planner.run(pg, neo4j, entity_id)
5. synthesizer.synthesize(pg, question, plan_result, intent)
6. return QueryResponse
```

`intent.py` — `classify_intent(question)`

Two-stage classification:

Stage 1 — Heuristic rules (fast, no LLM cost):
- `CAUSAL_WHY` — keywords: why, cause, reason, lead to, result in, because, due to
- `TEMPORAL_RANGE` — keywords: when, during, between, timeline, period, from … to
- `ENTITY_TIMELINE` — keywords: history of, timeline of, evolution of, over time
- `SIMILARITY` — fallback for any unmatched question

Stage 2 — LLM fallback (only if heuristics score is low):
Calls Ollama with `INTENT_CLASSIFICATION` prompt template, parses JSON response.

Returns `IntentResult(intent: str, confidence: float, method: "heuristic"|"llm")`.

`temporal_extractor.py` — `extract_time_range(question)`

Uses the NLP `temporal_parser` to find date spans in the question text, then collapses them into a single `TimeRange(start, end)` spanning all found spans. Returns `None` if no temporal expressions are found.

`entity_resolver.py` — `resolve_entity_filter(session, question)`

Extracts potential entity mentions from the question, then runs a three-tier resolution against PostgreSQL:

Tier	Method	Confidence
1	Exact `canonical_name` match (case-insensitive)	1.0
2	Match in `aliases` JSONB array	0.95
3	rapidfuzz `token_set_ratio` ≥ 75	0.75+

Returns `UUID` of the matched entity or `None`.

Query Planners — `app/query/planners/`

All planners return `PlanResult(events, causal_chain, document_ids, confidence)`.

`causal_planner.py`
- Embeds the question → pgvector similarity search → seed events
- For each seed: `graph_store.get_causal_chain(seed_id, max_hops)`

- Deduplicates chain records by `event_id`; sorts by `hop`
- Fetches chain event details from PostgreSQL
- `confidence = min(0.90, 0.70 + 0.10 × len(unique_chain_records))`

`temporal_planner.py`

- Calls `event_store.list_events(from_date, to_date, entity_id)`
- `confidence = 0.85` if `time_range` provided, else `0.60`
- `causal_chain = []` always

`similarity_planner.py`

- Embeds question → `event_store.similarity_search(embedding, k=limit)`
- Optional post-filters: entity_id (fetches linked event IDs from PG), time_range (filters by `ts_start`)
- Trims to `limit` after filtering
- `confidence = mean(1 - distance)` across results; `0.0` if empty
- `causal_chain = []` always

`entity_planner.py`

- Returns empty plan if `entity_id` is None
- `event_store.list_events(entity_id=entity_id)` for PostgreSQL events
- `graph_store.get_entity_graph(entity_id)` for Neo4j subgraph
- Builds `causal_chain` from Neo4j edge `cause_id` and `effect_id`; deduplicates
- `confidence = 0.88` if events found, else `0.0`

`synthesizer.py` — `synthesize(pg, question, plan, intent)`

1. Converts `plan.events` to `EventBrief` list
2. Converts `plan.causal_chain` to `CausalChainLink` list
3. Fetches `Document` rows for all `plan.document_ids` from PostgreSQL
4. Calls `OllamaClient.generate()` with `ANSWER_SYNTHESIS` prompt
5. Falls back to `_fallback_answer(question, events, chain)` if Ollama fails
6. Returns full `QueryResponse(answer, confidence, intent, causal_chain, events, sources)`

5.9 LLM Integration — `app/llm/`

`client.py` — `OllamaClient`

Async `httpx` client targeting `http://localhost:11434` (configurable via `OLLAMA_BASE_URL`).

Method	Ollama endpoint	Use
`generate(prompt, model)`	`POST /api/generate`	Single-turn completion
`chat(messages, model)`	`POST /api/chat`	Multi-turn conversation
`embed(text, model)`	`POST /api/embeddings`	Single text embedding
`embed_batch(texts, model)`	`POST /api/embeddings` × N	Batch embedding
`generate_code(prompt)`	`POST /api/generate` (codellama)	Cypher/SQL generation
`is_healthy()`	`GET /`	Liveness probe

Default synthesis model: `llama3.1:8b`. Default codegen model: `codellama:7b`.
Request timeout: 120 seconds.

`prompts.py`

Template constant	Used by
`INTENT_CLASSIFICATION`	`intent.py` LLM fallback
`ANSWER_SYNTHESIS`	`synthesizer.py`
`EVENT_EXTRACTION`	Future/alternative extraction path
`CAUSAL_EXTRACTION`	Future/alternative extraction path
`CYPHER_GENERATION`	Future graph query generation
`SQL_GENERATION`	Future SQL query generation
`COREF_RESOLUTION`	Future LLM-based coreference

5.10 API Layer — `app/api/`

Authentication — `app/api/routes/__init__.py`

Every route (except `GET /health`) requires the header:

```
X-API-Key: <value of API_KEY setting>
```

Default: `changeme`. The dependency `require_api_key` raises `HTTP 401` if the header is missing or incorrect.

`middleware.py` — `RequestLoggingMiddleware`

Logs every request (excluding `/health`, `/docs`, `/redoc`, `/openapi.json`) using structlog with fields: `method`, `path`, `status_code`, `duration_ms`.

6. Database Schema

See Section 5.4 for full column listings. The schema is created by a single Alembic migration (`c73d4a0b0c3e_initial_tables.py`) which:

1. Creates the `pgvector` extension
2. Creates tables in dependency order: `documents` → `entities` → `events` → `event_entities` → `causal_relations`
3. Creates all indexes including the HNSW vector index on `events.embedding`

---

7. API Reference

Base URL: `http://localhost:8000`
Authentication: `X-API-Key: <API_KEY>` header on all endpoints except `/health`.

`GET /health`

Health check. No authentication required.

Response 200:

```
{
  "status": "healthy",
  "app": "TemporalDB",
  "version": "0.1.0"
}
```

`POST /ingest`

Ingest raw text. Normalises, deduplicates, persists to PostgreSQL, publishes to Kafka.

Request body:

```
{
  "text": "Inflation rose sharply in Q3 2024 due to supply chain disruptions.",
  "source": "Reuters",
  "metadata": { "author": "Jane Doe" }
}
```

Response 202 (new document, queued for NLP processing):

```
{
  "document_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "source": "Reuters",
  "filename": null,
  "status": "processing",
  "message": "Document queued for NLP processing."
}
```

Response 200 (duplicate detected via Redis):

```
{
  "document_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "source": "Reuters",
  "filename": null,
  "status": "duplicate",
  "message": "Document already ingested."
}
```

`POST /ingest/file`

Ingest a file. Supports PDF, DOCX, TXT, and Markdown.

Request: `multipart/form-data`
- `file` — the uploaded file
- `source` (optional) — label for the document source

Response: same structure as `POST /ingest`.

`GET /events`

Paginated list of extracted events.

Query parameters:

Parameter	Type	Description
`skip`	int	Pagination offset (default: 0)
`limit`	int	Page size, max 100 (default: 20)
`entity_id`	UUID	Filter to events linked to this entity
`document_id`	UUID	Filter to events from this document
`from_date`	datetime	Events with `ts_start >= from_date`
`to_date`	datetime	Events with `ts_start <= to_date`
`event_type`	string	Filter by event type

Response 200:

```
{
  "items": [
    {
      "id": "...",
      "description": "Supply chains disrupted.",
      "event_type": "state_change",
      "ts_start": "2024-07-01T00:00:00Z",
      "ts_end": null,
      "confidence": 0.87,
      "source_sentence": "Supply chains were severely disrupted …",
      "document_id": "..."
    }
  ],
  "total": 142,
  "skip": 0,
  "limit": 20
}
```

`GET /events/{event_id}`

Single event by UUID.

Response 200: full `EventOut` object (same fields as above).
Response 404: event not found.

`GET /entities`

Paginated list of extracted entities.

Query parameters:

Parameter	Type	Description
`skip`	int	Pagination offset (default: 0)
`limit`	int	Page size, max 100 (default: 20)
`name`	string	Substring search on `name` or `canonical_name`
`entity_type`	string	Filter by NER label (PERSON, ORG, GPE, ...)

Response 200:

```
{
  "items": [
    {
      "id": "...",
      "name": "Federal Reserve",
      "canonical_name": "federal reserve",
      "type": "ORG",
      "aliases": ["the Fed", "Federal Reserve Board"],
      "created_at": "2024-08-01T10:00:00Z"
    }
  ],
  "total": 38,
  "skip": 0,
  "limit": 20
}
```

`GET /entities/{entity_id}`

Single entity by UUID.

Response 200: full `EntityOut` object.
Response 404: entity not found.

`GET /graph/entity/{entity_id}`

Causal subgraph centred on an entity. Returns all events involving the entity plus all CAUSES edges among those events. Suitable for D3.js / Cytoscape rendering.

Query parameters:

Parameter	Type	Description
`max_events`	int	Max event nodes to return (default: 50, max: 200)

Response 200:

```
{
  "entity": {
    "id": "...",
    "name": "Federal Reserve",
    "canonical_name": "federal reserve",
    "type": "ORG"
  },
  "nodes": [
    { "id": "evt-uuid-1", "label": "Interest rates raised", "type": "event",
      "ts_start": "2024-03-20T00:00:00Z", "confidence": 0.91 },
    { "id": "evt-uuid-2", "label": "Mortgage costs increased", "type": "event",
      "ts_start": "2024-04-01T00:00:00Z", "confidence": 0.85 }
  ],
  "edges": [
    { "source": "evt-uuid-1", "target": "evt-uuid-2",
      "type": "CAUSES", "confidence": 0.80, "evidence": "led to" }
  ]
}
```

Response 404: entity not found.

`POST /query`

Natural language query. Classifies intent, runs the appropriate planner, and returns a synthesised answer with citations.

Request body:

```
{
  "question": "Why did mortgage costs increase in early 2024?",
  "entity_filter": "Federal Reserve",
  "time_range": {
    "start": "2024-01-01T00:00:00Z",
    "end": "2024-06-30T23:59:59Z"
  },
  "max_causal_hops": 3,
  "limit": 10
}
```

All fields except `question` are optional.

Response 200:

```
{
  "answer": "Mortgage costs increased in early 2024 primarily because the Federal Reserve raised interest rates …",
  "confidence": 0.87,
  "intent": "CAUSAL_WHY",
  "causal_chain": [
    {
      "event_id": "evt-uuid-1",
      "description": "Interest rates raised",
      "ts_start": "2024-03-20T00:00:00Z",
      "hop": 1,
      "confidence": 0.91
    }
  ],
  "events": [
    { "id": "evt-uuid-1", "description": "Interest rates raised", "ts_start": "…" }
  ],
  "sources": [
    { "document_id": "doc-uuid-1", "source": "Reuters", "filename": null }
  ]
}
```

---

8. Data Flow Walkthroughs

Ingestion — Full End-to-End

```
1. Client sends POST /ingest { text, source }
2. Route handler:
     a. normalize(text)
     b. compute_fingerprint(text) → hash
     c. redis.get("dedup:doc:<hash>")
        → if hit: return 200 {status: "duplicate", document_id: existing_id}
     d. INSERT INTO documents (source, raw_text, content_hash, …)
     e. redis.set("dedup:doc:<hash>", document_id)
     f. publish_document_ingested(producer, document_id, source)
     g. return 202 {status: "processing"}

3. NLP Worker receives Kafka message:
     a. Fetch document text from PostgreSQL
     b. run_pipeline_sync(raw_text) → PipelineResult
     c. For each linked_entity:
          upsert_entity(pg, neo4j, name, canonical_name, type)
     d. For each extracted_event:
          embed(event.description) → vector
          insert_event(pg, description, embedding, ts_start, …)
          link_entities_to_event(pg, event_id, entity_ids)
          upsert_event_node(neo4j, event_id, …)
     e. For each entity:
          upsert_involves_edge(neo4j, event_id, entity_id)
     f. For each causal_relation:
          insert_causal_relation(pg, cause_id, effect_id, …)
          upsert_causal_edge(neo4j, cause_id, effect_id, relation_id, …)
```

Query — CAUSAL_WHY

```
1. Client sends POST /query { question: "Why did X happen?" }
2. classify_intent → CAUSAL_WHY (heuristic "why")
3. extract_time_range → None (no date in question)
4. resolve_entity_filter → None (no entity mention)
5. causal_planner.run(pg, neo4j, question, max_hops=3):
     a. embed(question) → query_vector
     b. similarity_search(pg, query_vector, k=5) → [(event, distance), …]
     c. for each seed_event:
          graph_store.get_causal_chain(neo4j, seed_id, direction="downstream")
     d. deduplicate + sort chain by hop
     e. fetch chain event details from PG
     f. confidence = min(0.90, 0.70 + 0.10 × chain_length)
6. synthesizer.synthesize(pg, question, plan, "CAUSAL_WHY"):
     a. fetch Documents for plan.document_ids
     b. OllamaClient.generate(ANSWER_SYNTHESIS prompt)
        → on failure: _fallback_answer(question, events, chain)
     c. build QueryResponse
7. return QueryResponse
```

---

9. Running Locally

Prerequisites

Install the following before starting:

Tool	Version	Install
Python	3.12	python.org or `brew install python@3.12`
Docker Desktop	latest	docker.com
Ollama	latest	ollama.ai or `brew install ollama`

Step 1 — Navigate to the project directory

```
cd "/Users/softwareengineer/Desktop/TemporalDB/TemporalDB Backend"
```

Step 2 — Create and activate a virtual environment

```
python3.12 -m venv temporalenv
source temporalenv/bin/activate
```

Step 3 — Configure environment variables

```
cp .env.example .env
```

The defaults in `.env.example` match the `docker-compose.yml` ports exactly, so no changes are required to run locally. Review and update:

Setting	Default	Change if...
`API_KEY`	`changeme`	Always change in production
`DEBUG`	`false`	Set `true` for verbose error messages during development
`OLLAMA_SYNTHESIS_MODEL`	`llama3.1:8b`	You want a different model

Step 4 — Start Docker infrastructure services

```
make up
# or: docker compose up -d
```

This starts five containers:

Container	Service	Port
`temporaldb-postgres`	PostgreSQL 16 + pgvector	`5433`
`temporaldb-neo4j`	Neo4j 5 Community	`7475` (HTTP), `7688` (Bolt)
`temporaldb-redis`	Redis 7	`6380`
`temporaldb-redpanda`	Redpanda (Kafka)	`29092`
`temporaldb-pgadmin`	pgAdmin 4	`5051`

Wait for all services to pass their health checks:

```
make ps
# Wait until all STATUS values show "healthy"
```

Step 5 — Install Python dependencies

```
make install
# or: pip install -r requirements.txt
```

Step 6 — Download the spaCy language model

```
make spacy-download
# or: python -m spacy download en_core_web_trf
```

Note: `en_core_web_trf` is the transformer-based pipeline (~500 MB).
For faster local development, download the small model instead:
`bash make spacy-download SPACY_MODEL=en_core_web_sm`
and update `SPACY_MODEL=en_core_web_sm` in your `.env` file.

Step 7 — Pull Ollama models

Make sure the Ollama application is running, then pull the required models:

```
ollama pull llama3.1:8b        # answer synthesis
ollama pull codellama:7b        # code / Cypher generation (optional)
```

Verify Ollama is reachable:

```
curl http://localhost:11434
# Expected: "Ollama is running"
```

Step 8 — Run database migrations

```
make migrate
# or: alembic upgrade head
```

This creates the following tables in PostgreSQL:
`documents`, `entities`, `events`, `event_entities`, `causal_relations`
plus the `pgvector` extension and all indexes.

Verify with pgAdmin at `http://localhost:5051`
(email: `admin@temporaldb.local`, password: `admin`).

Step 9 — Start the API server

```
make run
# or: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

The API is now available at `http://localhost:8000`.

Verify:

```
curl http://localhost:8000/health
# {"status":"healthy","app":"TemporalDB","version":"0.1.0"}
```

Interactive API docs: `http://localhost:8000/docs`

Step 10 — Start the NLP worker (separate terminal)

The NLP worker consumes from the Kafka topic and runs the full pipeline on ingested documents. Open a new terminal:

```
source temporalenv/bin/activate
make worker
# or: python -m workers.nlp_worker
```

Step 11 — Ingest a document and run a query

Ingest text:

```
curl -X POST http://localhost:8000/ingest \
  -H "X-API-Key: changeme" \
  -H "Content-Type: application/json" \
  -d '{
    "text": "The Federal Reserve raised interest rates in March 2024, which led to higher mortgage costs across the
    "source": "Test"
  }'
```

Wait a few seconds for the NLP worker to process the document, then:

Run a query:

```
curl -X POST http://localhost:8000/query \
  -H "X-API-Key: changeme" \
  -H "Content-Type: application/json" \
  -d '{"question": "Why did mortgage costs increase?"}'
```

Quick Reference — All `make` Commands

```
make setup           Bootstrap: copy .env.example → .env, install deps
make up              Start Docker services
make down            Stop Docker services
make logs            Tail service logs
make ps              Show service status
make db-shell        Open psql inside the postgres container

make migrate         Apply all Alembic migrations
make migration       Generate migration: make migration msg="add column"
make migrate-down    Roll back one migration
make migrate-status  Show current migration revision

make run             Start API server with hot-reload (port 8000)
make worker          Start NLP Kafka worker

make test            Run unit + API tests
make test-cov        Run tests with coverage report
make lint            ruff check
make fmt             ruff format

make docker-build    Build production Docker image
make docker-run      Run API in Docker with .env

make clean           Remove __pycache__, .coverage, htmlcov
```

Service URLs at a Glance

Service	URL
API	http://localhost:8000
API docs (Swagger)	http://localhost:8000/docs
API docs (ReDoc)	http://localhost:8000/redoc
pgAdmin	http://localhost:5051
Neo4j Browser	http://localhost:7475
Ollama	http://localhost:11434
Redpanda Admin	http://localhost:9645

Troubleshooting

Docker services not healthy:

```
docker compose logs postgres   # check PostgreSQL startup
docker compose logs neo4j      # Neo4j takes ~30s to start
```

`alembic upgrade head` fails:
Ensure PostgreSQL is healthy before running migrations.
Check `POSTGRES_HOST/PORT` in `.env` match the docker-compose port mapping (`5433`).

NLP worker not processing documents:
Confirm Redpanda is healthy (`make ps`) and the worker is running.
Check that `KAFKA_BOOTSTRAP_SERVERS=localhost:29092` in `.env`.

Ollama timeout on first query:
The first call loads the model into GPU/CPU memory and can take 30–60 seconds.
Subsequent queries will be faster. Increase `OLLAMA_REQUEST_TIMEOUT` in `.env`
if needed.

spaCy model not found:

```
python -m spacy validate        # list installed models
make spacy-download             # re-download en_core_web_trf
```